



智能移动技术 实验报告

课程名称 智能移动技术

姓 名 XXX

日 期 2025 年 4 月 1 日

指导老师 王越、熊蓉

一、 原理分析

1.1 ICP 算法

本次实验采用 ICP(Iterative Closest Point) 算法，具体的流程如下：

- 估计 P' 集合点与 P 集合点的初始位姿关系
- 根据最近邻域规则建立 P' 集合点与 P 集合点的关联
- 利用线性代数/非线性优化的方式估计旋转平移量
- 对点集合 P' 的点进行旋转平移
- 如果旋转平移后重新关联的均方差小于阈值，则结束
- 否则迭代重复上述步骤

1.2 点对点 ICP

算法的输入为两个点集合

$$A = \{a_1, \dots, a_n\}, B = \{b_1, \dots, b_n\}$$

目标是计算两组数据之间的旋转平移量 R, t ，使得两组数据形成最佳匹配，即两组数据的距离误差最小

第 i 个匹配对点的误差为

$$e_i = a_i - (Rb_i + t)$$

构建成最小二乘问题，求使得误差平方和达到最小的 R, t

$$\min \frac{1}{2} \sum_{i=1}^n \|a_i - (Rb_i + t)\|^2$$

定义两组点集合的质心位置 a, b

$$a = \frac{1}{n} \sum_{i=1}^n a_i, b = \frac{1}{n} \sum_{i=1}^n b_i$$

计算每个点的去质心坐标

$$q_i = a_i - a, q'_i = b_i - b$$

根据以下优化问题计算旋转矩阵

$$R^* = \arg \min \frac{1}{2} \sum_{i=1}^n \|q_i - Rq'_i\|^2$$

根据 R^* 可计算 t^*

$$t^* = a - R^*b$$

其中，矩阵 R^* 可由 SVD 分解计算得到：

定义矩阵

$$W = \sum_{i=1}^n q'_i q_i^T$$

对 W 进行 SVD 分解，得

$$W = USV^T$$

则 R^* 可由下式计算

$$R^* = VU^T$$

二、算法改进

2.1 迭代终止判断

在代码中，设置了最大迭代次数，但是在代码实际运行时发现，有些相邻两帧点云进行相互匹配时，并不需要达到最大迭代次数，其差值就已经很小，因此，在迭代循环中，添加了一个循环终止的判断，通过判断两次误差的差值是否超过阈值来终止迭代。

```
1 % 计算本次迭代匹配对的总误差
2 err = 0;
3 for i = 1:numKeep
4     err = err + norm(targetFiltered(:, i) - sourceFiltered(:, i));
5 end
6 if abs(err - err_prev) < 1e-6
7     break;
8 end
```

2.2 删除部分点云

按照上面的 Point to Point 算法实现后，发现与 Matlab 的 ICP 结果差值较大，点云匹配之间也有较大误差。鉴于这种情况，考虑可能是因为在进行临近点搜索时距离较远的点对算法的影响较大，因此，通过删除部分的点云，来提高算法的精确度

```
1 % 根据保留比例删除距离较大的匹配对
2 % reserveRatio:保留比例
3 numKeep = floor(numPts * reserveRatio);
4 [~, sortIdx] = sort(distances);
5 targetFiltered = matchedTarget(:, sortIdx(1:numKeep));
6 sourceFiltered = sourcePts(:, sortIdx(1:numKeep));
```

2.3 最终 ICP 算法函数

我们使用 MATLAB 实现了 ICP 算法，以下是算法的主要步骤：

1. 初始化：设置最大迭代次数，并提取点云数据。
2. 第一次迭代：计算初始质心，并进行第一次旋转和平移。
3. 迭代过程：
 - 对于每个源点，找到目标点云中距离最近的点。
 - 根据匹配结果，重新排列目标点云的对应点。
 - 根据保留比例删除距离较大的匹配对。
 - 计算选中点集的质心，并进行去质心化。
 - 计算协方差矩阵，并使用 SVD 分解求最优旋转矩阵。
 - 求最优平移向量，并更新源点云坐标。
 - 计算本次迭代匹配对的总误差，如果误差变化小于阈值，则结束迭代。
4. 输出结果：输出旋转矩阵、平移向量和变换后的点云。

具体的函数如下所示

```
1 function [R_f, t_f, p_transformed] = ICP(targetPC, sourcePC,  
    reserveRatio)  
2 % 设置最大迭代次数  
3 maxIter = 100;  
4  
5 % 提取点云数据，每个点作为列向量  
6 targetPts = targetPC.Location';  
7 sourcePts = sourcePC.Location';  
8  
9 % 点云点数  
10 numPts = length(targetPts);  
11  
12 % 第一次迭代  
13 % 计算初始质心  
14 centerTarget = mean(targetPts, 2);  
15 centerSource = mean(sourcePts, 2);  
16  
17 % 初始参数  
18 R_current = eye(3);  
19 R_f = eye(3);
```

```

20     t_current = centerTarget - centerSource;
21     t_f = t_current;
22
23     % 第一次旋转结果
24     sourcePts = R_current * sourcePts + t_current;
25
26     % 迭代参数初始化
27     iteration = 0;
28     err_prev = inf;
29
30     for iteration = 1 : maxIter
31         % 对于每个源点，找到目标点云中距离最近的点
32         distances = zeros(1, numPts);
33         indices = zeros(1, numPts);
34         for i = 1:numPts
35             diff = targetPts - repmat(sourcePts(:, i), 1, numPts);
36             dists = sqrt(sum(diff.^2, 1));
37             [distances(i), indices(i)] = min(dists);
38         end
39
40         % 根据匹配结果，重新排列目标点云的对应点
41         matchedTarget = targetPts(:, indices);
42
43         % 根据保留比例删除距离较大的匹配对
44         numKeep = floor(numPts * reserveRatio);
45         [~, sortIdx] = sort(distances);
46         targetFiltered = matchedTarget(:, sortIdx(1:numKeep));
47         sourceFiltered = sourcePts(:, sortIdx(1:numKeep));
48
49         % 计算选中点集的质心
50         centerTarget_filtered = mean(targetFiltered, 2);
51         centerSource_filtered = mean(sourceFiltered, 2);
52
53         % 去质心化
54         targetZeroMean = targetFiltered - centerTarget_filtered;

```

```

55     sourceZeroMean = sourceFiltered - centerSource_filtered;
56
57     % 计算协方差矩阵
58     W = zeros(3, 3);
59     for i = 1:numKeep
60         W = W + sourceZeroMean(:, i) * targetZeroMean(:, i)';
61     end
62
63     % SVD 分解求最优旋转
64     [U, ~, V] = svd(W);
65     R_current = V * U';
66
67     % 求最优平移
68     t_current = centerTarget_filtered - R_current * centerSource_
        filtered;
69
70     % 累积变换
71     R_f = R_current * R_f;
72     t_f = R_current * t_f + t_current;
73
74     % 更新源点云坐标
75     sourcePts = R_current * sourcePts + t_current;
76
77     % 计算本次迭代匹配对的总误差
78     err = 0;
79     for i = 1:numKeep
80         err = err + norm(targetFiltered(:, i) - sourceFiltered(:, i
            ));
81     end
82     if abs(err - err_prev) < 1e-6
83         break;
84     end
85     err_prev = err;
86 end
87 % disp(iteration);

```

```
88
89 % 输出旋转矩阵、平移向量和变换后的点云
90 p_transformed = pointCloud(sourcePts');
91 end
```

三、结果展示

在 */src/result* 文件夹里有相邻两帧点云的匹配前后的图片，下面是最后两帧的结果显示

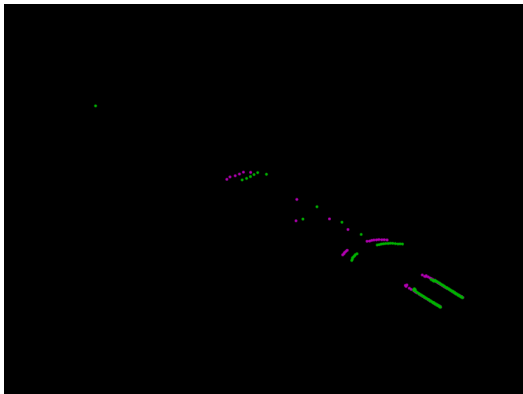


图1 匹配前

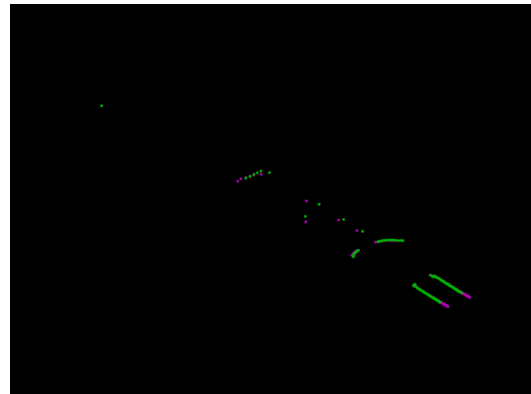
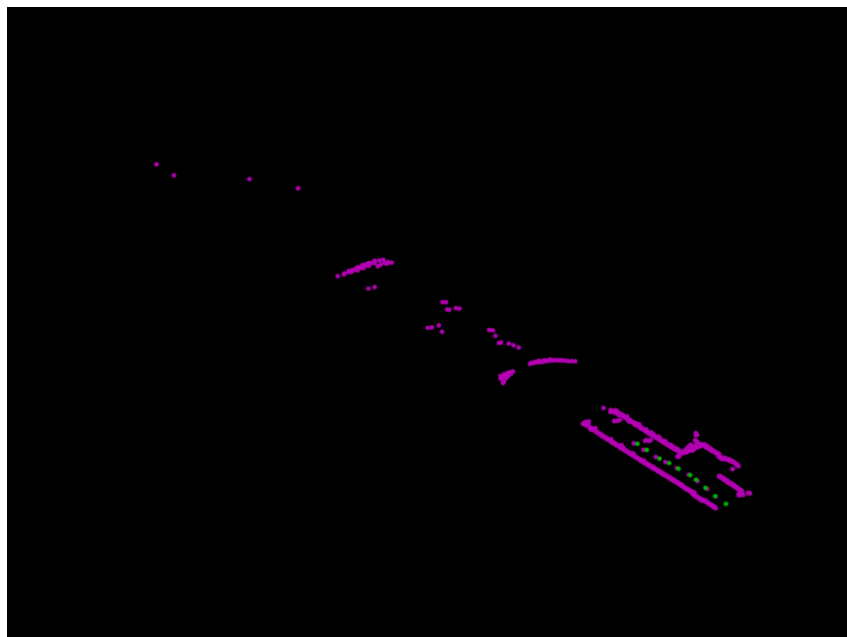


图2 匹配后

最终的轨迹以及融合的点云如下图所示 (中间点绿色代表 Matlab 的轨迹，粉色的代表自己 ICP 的轨迹以及总的融合的点云)



最终自己 ICP 的算法与 Matlab 算法的差值如下，为 **1.6113m**。

```
命令行窗口
轨迹总误差： 1.6113
fx >>
```

四、结果分析

4.1 误差分析

根据实验结果,我们自己实现的 ICP 算法与 Matlab 算法之间的距离误差为 1.6113m。这一误差可能由以下几个因素引起:

- 算法实现细节的差异: 尽管我们参考了标准的 ICP 算法原理,但在具体实现过程中可能存在一些细节上的差异,如数据预处理、点云匹配策略等。
- 点云数据的噪声: 实际的点云数据通常包含噪声,这可能会影响算法的配准精度。
- 参数设置的影响: ICP 算法的性能受到一些参数的影响,如保留比例、误差阈值等。不同的参数设置可能会导致不同的配准结果。

4.2 改进思路

通过本次实验,我对 ICP 算法有了更深入的理解,但也意识到 ICP 算法对噪声的敏感性以及对初始位姿的依赖等。为了减小误差,可以从以下方式进行改进:

- 结合其他配准方法: 将 ICP 算法与其他配准方法(如 Line to line)相结合,以充分利用不同方法的优势,提高配准的准确性和鲁棒性。
- 优化参数设置: 通过实验和理论分析,进一步优化 ICP 算法的参数设置,以获得更好的配准效果

附录 A 文件列表

文件名	功能描述
data	实验数据与示例
src/result	实验结果
src/icp_map.m	相邻两帧点云匹配与结果绘制
src/traj_map.m	融合点云及轨迹结果和绘制
report.pdf	实验报告
README.md	文件简介